

Enfflash: Truly Real-Time Enforcement of First-Order Temporal Requirements

Abstract—Runtime enforcers observe a system’s behavior and issue corrections to ensure that the system always adheres to its specification. Access-control engines, security kernels, and application firewalls are all instances of runtime enforcers. In fields such as software security or data protection, many natural requirements involve temporal sequences of data-carrying events. This is the case, e.g., when user consent must be registered before an action is performed or when data must be erased by some deadline. Real-world specifications may include hundreds of such requirements. However, existing enforcers supporting such complex temporal specifications are often too slow to be used in production software. Alternatively, developers may resort to non-temporal permission engines and ad-hoc instrumentation, but these scale poorly with complex temporal specifications.

To address this gap, we introduce an efficient algorithm and tool for enforcing complex temporal specifications in software. We identify a fragment of Metric First-Order Temporal Logic (MFOTL) that can be enforced with low, predictable runtime complexity, while supporting a rich family of practically relevant specifications. We present a new tool, called Enfflash, that enforces specifications in this fragment with latencies $3\text{--}20\times$ lower than the previous state-of-the-art. This performance is achieved by compiling enforceable specifications into imperative programs for which we develop an efficient interpreter.

We evaluate Enfflash on both existing benchmarks (in isolation) and as an instrumentation backend for web applications, demonstrating that our tool can enforce complex specifications with hundreds of rules with $1\text{--}20$ ms latency.

Index Terms—runtime enforcement, security, privacy, logic

I. INTRODUCTION

Modern software must comply with an ever-growing body of requirements, including access-control rules, security policies, and data-protection regulations. Many of these requirements are inherently temporal and carry data (e.g., identifiers): user consent must be recorded before their data is used, erasure of old messages must occur within a deadline, an account must be locked after a sequence of failed logins, etc. Rather than auditing requirements *ex post*, *runtime enforcement* [1], [2] observes the system’s behavior and corrects it as it runs, suppressing or causing actions to ensure compliance.

Enforcing temporal, data-carrying policies is the focus of enforcers based on Metric First-Order Temporal Logic (MFOTL) [3], such as EnfGuard [4] and WhyEnf [5]. Unfortunately, the latency of these tools is high and may grow with the size of the enforcer’s state. Typically, these enforcers induce a latency of a $10\text{--}1000$ ms even for relatively small policies [4], [6], making them unsuitable to instrument typical user-facing functions that must respond within $10\text{--}100$ ms. In contrast, non-temporal permission engines such as Cedar [7] and ad-hoc instrumentation promise sub-millisecond runtime

performance. However, the former are not designed for policies that depend on previously recorded actions, while the latter is brittle and scales poorly: a real policy may comprise of hundreds of interacting requirements, and encoding their temporal dependencies by hand is error-prone and hard to maintain. To improve upon the status quo, software developers need a tool that is both expressive enough to support first-order temporal policies and scalable enough for production use.

In this paper, we introduce Enfflash, a new tool for enforcing first-order temporal specifications with low, predictable latency. Unlike previous tools [4], [5], [8], Enfflash first *compiles* MFOTL formulae into imperative programs that describe an enforcer for the corresponding formula, and then executes these programs. The generated programs can be efficiently evaluated at runtime, and their complexity can be estimated statically. The MFOTL fragment supported by Enfflash is large enough to express many access-control and data-protection requirements that arise in practice, including requirements involving aggregations and user-defined Python functions.

We evaluate our tool’s expressivity and performance against existing benchmarks [4], [7] and deploy it as an enforcement backend for two middle-sized web applications: a social network platform that is instrumented for compliance with a 400-line specification of the General Data Protection Regulation (GDPR), and an event management platform that comes with a broad range of access-control rules. Overall, we measure latencies $3\text{--}20\times$ lower than the previous state of the art, resulting in a modest $1\text{--}20$ ms runtime cost that is compatible with deployment in many user-facing systems.

After reviewing enforcement preliminaries (Section II), we make the following contributions:

- We design EF, a domain-specific imperative language that captures the behavior of a proactive first-order enforcer, give it a formal semantics, and show that EF programs execute with predictable complexity (Section III).
- We introduce a compilation procedure from MFOTL to EF, generating sound enforcers for an expressive fragment of MFOTL (Section IV).
- We implement our algorithms in a new tool, called Enfflash, and evaluate it both on existing enforcement benchmarks and as an instrumentation backend for web applications. We show that Enfflash enforces specifications with hundreds of rules with modest latency, achieving a $10\text{--}20\times$ speedup over previous tools (Section V).

Finally, we review related work (Section VI) and conclude.

II. RUNTIME ENFORCEMENT

We first introduce traces that model system executions, metric-first order temporal logic (MFOTL), and enforcers.

A. Traces

Let $\mathbb{E}$ be a finite set of *event names* with arity function $\iota : \mathbb{E} \rightarrow \mathbb{N}$. An *event* is a pair $(e, \vec{d}) \in \mathbb{E} \times \mathbb{D}^{\iota(e)}$ of an event name e and its $\iota(e)$ arguments, tuple $\vec{d}$. A *signature* $\Sigma = (\mathbb{D}, \mathbb{E}, \iota)$ defines the set of *databases* $\mathbb{DB} := \mathcal{P}(\{(e, \vec{d}) \mid e \in \mathbb{E}, \vec{d} \in \mathbb{D}^{\iota(e)}\})$. For $E \subseteq \mathbb{E}$, let $\mathbb{DB}_E := \{D \in \mathbb{DB} \mid \forall (e, \vec{d}) \in D. e \in E\}$ denote databases with event names in E .

Example 1. Consider a file management system. Files can be accessed (‘used’) or deleted, and users can request deletion of their data, give consent for files of some category to be used, or revoke said consent. We model this with the signature $\Sigma_0 = (\mathbb{N}, \mathbb{E}_0, \iota_0)$, where $\mathbb{E}_0 = \{\text{use}, \text{consent}, \text{revoke}, \text{delete}, \text{deletion_request}\}$, $\iota_0(\text{use}) = 3$, and $\iota_0(\text{delete}) = \iota_0(\text{deletion_request}) = \iota_0(\text{consent}) = \iota_0(\text{revoke}) = 2$. The event $\text{use}(c, d, u)$ denotes ‘system uses user u ’s data d from category c ’, $\text{delete}(c, d)$ denotes ‘user u ’s data d is deleted’, $\text{deletion_request}(d, u)$ denotes ‘user u requests deletion of data d ’, $\text{consent}(u, c)$ denotes ‘user u provides consent for category c ’, and $\text{revoke}(u, c)$ denotes ‘user u revokes consent for category c ’.

A trace σ is a sequence $\langle (\tau_i, D_i) \rangle_{0 \leq i \leq k}, k \in \mathbb{N} \cup \{\infty\}$ of pairs of timestamps $\tau_i \in \mathbb{N}$ and finite databases $D_i \in \mathbb{DB}$, where timestamps grow monotonically ($\forall i < |\sigma|. \tau_i \leq \tau_{i+1}$) and progress (if $|\sigma| = \infty$, then $\forall \tau. \exists i. \tau < \tau_i$). An index $0 \leq i < |\sigma|$ in a trace σ is called a *time-point*. The empty trace is denoted by ε , the set of all traces by $\mathbb{T}$.

Example 2. Consider two traces of the above system:

$$\begin{aligned} \sigma_1 &= (10, \{\text{consent}(1, 1)\}), (50, \{\text{use}(1, 3, 1), \text{use}(2, 1, 1)\}), \dots \\ \sigma_2 &= (10, \{\text{deletion_request}(1, 1)\}), (50, \{\text{use}(1, 3, 1)\}), \dots \end{aligned}$$

In σ_1 , user 1 provides consent for category 1 at time-point 0 with timestamp 10; at time-point 1 with timestamp 50, the system uses user 1’s data 3 (with category 1) and user 1’s data 1 (with category 2). In σ_2 , user 1 requests deletion of data 1, and then the system uses data 3 with category 1.

B. Metric first-order temporal logic (MFOTL)

MFOTL extends First-Order Logic (FOL) with temporal operators $\circ$ (‘next’), $\bullet$ (‘previous’), $\diamond$ (‘eventually’) and S (‘since’) [9] equipped with intervals [10] that encode constraints on timestamps. We also include let bindings that introduce intermediate definitions, as well as SQL-style and user-defined aggregations. We thus follow Hublet et al. [4], except the $\diamond$ operator replaces the U (‘until’) operator as a primitive, a deliberate restriction to the fragment we enforce.

Let us fix the following sets:

- $\mathbb{V}$ a set of variables;
- $\mathbb{F}$ a set of *function symbols* on $\mathbb{D}$, each $f \in \mathbb{F}$ of some arity n and interpreted by a fixed total function $\hat{f} : \mathbb{D}^n \rightarrow \mathbb{D}$ (e.g., arithmetic or string operations).

- Ω a set of *aggregation operator symbols* on sets of tuples of $\mathbb{D}^*$, each $\omega \in \Omega$ with arities (n, m) and interpreted by a fixed total function $\hat{\omega} : \mathcal{P}(\mathbb{D}^n) \rightarrow \mathcal{P}(\mathbb{D}^m)$.

The syntax of MFOTL formulae is defined as follows:

$$\begin{aligned} t &::= x \mid c \mid f(\vec{t}) \\ \varphi &::= \top \mid e(\vec{t}) \mid \neg \varphi \mid \varphi \wedge \varphi \mid \exists x. \varphi \mid \circ_I \varphi \mid \bullet_I \varphi \mid \diamond_I \varphi \\ &\quad \mid \varphi \text{S}_I \varphi \mid \text{let } e(\vec{x}) = \varphi \text{ in } \varphi \mid \vec{x} \leftarrow \omega(\vec{t}; \vec{x}) \varphi, \end{aligned}$$

where $e \in \mathbb{E}$, $x \in \mathbb{V}$, $c \in \mathbb{D}$, and $I \in \mathbb{I}$ ranges over non-empty intervals in $\mathbb{N}$. We use standard abbreviations $\perp := \neg \top$, $\varphi \vee \psi := \neg(\neg \varphi \wedge \neg \psi)$, $\varphi \rightarrow \psi := \neg \varphi \vee \psi$, $\varphi \leftrightarrow \psi := (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$, $\forall x. \varphi := \neg(\exists x. \neg \varphi)$, $\blacklozenge_I \varphi := \top \text{S}_I \varphi$ (‘once’), and $\square_I \varphi := \neg \diamond_I \neg \varphi$ (‘always’). and omit $[0, \infty)$ intervals from the temporal operators’ subscript. Furthermore, we write $\varphi[d/x]$ for the formula resulting from substituting the free variable x with the constant d in the formula φ .

Example 3. Suppose the time unit is days. Consider the following four formulae:

$$\begin{aligned} \varphi_{\text{law}} &= \square (\forall c, d, u. \text{use}(c, d, u) \rightarrow (\neg \text{revoke}(u, c) \text{S} \text{consent}(u, c))) \\ \varphi_{\text{del}} &= \square (\forall d, u. \text{deletion_request}(d, u) \rightarrow \diamond_{[0, 30]} \text{delete}(d, u)) \\ \varphi_{\text{let}} &= \text{let active_consent}(u, c) = \neg \text{revoke}(u, c) \text{S} \text{consent}(u, c) \text{ in} \\ &\quad \square (\forall c, d, u. \text{use}(c, d, u) \rightarrow \text{active_consent}(u, c)) \\ \varphi_{\text{agg}} &= n \leftarrow \text{CNT}(d; u) \blacklozenge_{[0, 10]} \text{use}(c, d, u) \end{aligned}$$

The first formula, φ_{law} , formalizes the lawfulness of processing: ‘whenever data d with category c belonging to user u is processed, then u must have consented to the use of category c at some point in the past and not since revoked that consent.’ The second formula, φ_{del} , formalizes the GDPR’s right to erasure: ‘whenever a user u requests the deletion of data d , then d must be deleted within 30 days.’ The third formula, φ_{let} , illustrates let bindings: it defines $\text{active_consent}(u, c)$ as holding when u has consented to category c and not revoked that consent, and uses this definition to restate φ_{law} more concisely. The fourth formula, φ_{agg} , illustrates aggregations: it counts, grouped by u , the number of data items of user u used over the last 10 time units.

We write $\text{fv}(\varphi)$ for the set of free variables of a formula φ . MFOTL’s formal semantics is defined in Figure 1. Given a valuation $v : \mathbb{V} \rightarrow \mathbb{D}$ whose domain includes $\text{fv}(v)$, we define the interpretation of terms as $\llbracket x \rrbracket_v = v(x)$ (for variables), $\llbracket c \rrbracket_v = c$ (for constants), and $\llbracket f(\vec{t}) \rrbracket_v = \hat{f}(\llbracket \vec{t} \rrbracket_v)$ (for function applications). Each sequent $v, i \models_\sigma \varphi$ denotes that φ is satisfied at time-point i of trace σ under valuation v . We omit σ whenever it is clear from the context.

C. Enforcers

We consider a *proactive* runtime enforcement model [5], [11] where non-compliant traces can be corrected in two ways: by modifying existing event databases by causing or suppressing events (*reactively*); or by inserting a single event database before any timestamp increment (*proactively*). The latter feature allows enforcers to satisfy real-time deadlines by inserting events. We assume that a fixed subset of all events

$$\begin{aligned}
v, i \models e(\bar{t}) &\text{ iff } (e, \llbracket \bar{t} \rrbracket_v) \in D_i \\
v, i \models \neg \varphi &\text{ iff } v, i \not\models \varphi \\
v, i \models \varphi \wedge \psi &\text{ iff } v, i \models \varphi \text{ and } v, i \models \psi \\
v, i \models \exists x. \varphi &\text{ iff } v[x \mapsto d], i \models \varphi \text{ for some } d \in \mathbb{D} \\
v, i \models \bigcirc_I \varphi &\text{ iff } v, i+1 \models \varphi \text{ and } \tau_{i+1} - \tau_i \in I \\
v, i \models \bullet_I \varphi &\text{ iff } i > 0 \text{ and } v, i-1 \models \varphi \text{ and } \tau_i - \tau_{i-1} \in I \\
v, i \models \Diamond_I \varphi &\text{ iff } v, j \models \varphi \text{ for some } j \geq i \text{ with } \tau_j - \tau_i \in I \\
v, i \models \varphi \text{ S}_I \psi &\text{ iff } v, j \models \psi \text{ for some } j \leq i \text{ with } \tau_i - \tau_j \in I \\
&\text{ and } v, k \models \varphi \text{ for all } j < k \leq i \\
v, i \models \bar{y} \leftarrow \omega(\bar{s}; \bar{g}) \varphi &\text{ iff } v(\bar{y}) \in \hat{G} \langle \llbracket \bar{s} \rrbracket_{v'} \mid v' \in \mathcal{G} \rangle
\end{aligned}$$

where $\mathcal{G} = \{v' \mid v'(\bar{g}) = v(\bar{g}), v', i \models \varphi\}$, $\langle \cdot \cdot \rangle$ is a multiset, and $\sigma[e \mapsto \varphi]$ extends each D_j of σ with $\{e(\llbracket \bar{x} \rrbracket_{v'}) \mid v', j \models \sigma \varphi\}$.

Fig. 1. MFOTL semantics for a fixed, infinite trace σ . The free variables of an aggregation (resp. table operator) are $\bar{g} \cup \{y\}$ (resp. $\bar{g} \cup \bar{o}$).

can be caused at any time ($\mathbb{C} \subseteq \mathbb{E}$), while another fixed subset can be suppressed ($\mathbb{S} \subseteq \mathbb{E}$). Formally, a *proactive enforcer* is a quadruple $\mathcal{E} = (\mathcal{S}, s_0, \mu, \nu)$ where $\mathcal{S}$ is a set of states, s_0 is an initial state, $\mu : \mathcal{S} \times \mathbb{T} \times \mathbb{N} \times \mathbb{DB} \rightarrow \mathcal{S} \times \mathbb{DB}_{\mathbb{C}} \times \mathbb{DB}_{\mathbb{S}}$ updates the state and causes and suppresses events when processing a database, and $\nu : \mathcal{S} \times \mathbb{T} \times \mathbb{N} \rightarrow \mathcal{S} \times (\mathbb{DB}_{\mathbb{C}} \cup \{\perp\})$ updates the state and optionally causes events (or does nothing) when a timestamp is incremented. As in [4], [5], the enforcer $\mathcal{E}$ modifies a trace ρ as follows:

Algorithm 1 Enforcement loop from Hublet et al. [5]

```

1: function  $\mathcal{E}(\rho = (\tau_i, D_i)_i)$ 
2:    $s \leftarrow s_0; \sigma \leftarrow \varepsilon$ 
3:   for  $i = 1, \dots, |\rho|$  do
4:      $(s, C, S) \leftarrow \mu(s, \sigma, \tau_i, D_i)$  ▷ reactive
5:      $\sigma \leftarrow \sigma \cdot (\tau_i, (D_i \setminus S) \cup C)$ 
6:     while  $\tau_i < \tau_{i+1}$  do
7:        $(s, C') \leftarrow \nu(s, \sigma, \tau_i)$  ▷ proactive
8:       if  $C' \neq \perp$  then  $\sigma \leftarrow \sigma \cdot (\tau_i, C')$  end if
9:        $\tau_i \leftarrow \tau_i + 1$ 
10:    end while
11:  end for
12:  return  $(s, \sigma)$ 
13: end function

```

The loop incrementally processes a trace of timestamped events (τ_i, D_i) , maintaining a current state s and an output trace σ . For each event, it first applies μ to update the state and transform the event's data (causing and suppressing event in C and S), then calls ν before any timestamp increment to proactively cause any additional events.

An enforcer $\mathcal{E}$ is *sound* with respect to a closed formula φ if for any system trace $\sigma \in \mathbb{T}$, we have $\emptyset, 0 \models_{\mathcal{E}(\sigma)} \varphi$, i.e., the modified trace $\mathcal{E}(\sigma)$ complies with φ .

III. EF: AN OPERATIONAL ENFORCEMENT LANGUAGE

We now present EF, a domain-specific language (DSL) that encodes the behavior of a proactive, first-order enforcer.

EF programs are trace transducers that cause and suppress events when specific triggers hold, store temporal relations into tables, and perform fixpoint computations. One key feature of EF programs is that their per-time-point evaluation complexity can be easily bounded in terms of the size of some of their internal tables. If these tables remain small, this provides guarantees that the program's latency will not significantly grow with trace size, a key feature for deployment in long-running systems. In Havelund et al.'s terminology [12], EF is an *operational specification language* with a concrete executable semantics. We present EF's syntax (Section III-A) and its semantics (Section III-B) and analyze the complexity of executing EF programs (Section III-C).

A. Syntax

We write *id* for identifiers, *n* for integer literals, and $\langle X \rangle$ for a comma-separated list of *X*. Figure 3 shows the full grammar of EF programs. Figure 2 shows the EF program enforcing the lawfulness policy φ_{law} of Example 3; we revisit it in Section IV, generating it automatically from φ_{law} .

An EF program is a sequence of *declarations* followed by a sequence of *items*. The declarations define the enforcement signature and helper code: **event** *idi*($\langle \tau \rangle$) declares an event with typed arguments; **pyinit** provides a Python preamble executed at start-up (imports and shared globals); **fun** declares a Python function usable inside terms; and **tfun** declares a custom aggregation operator mapping a list of rows to another. The items define the enforcer's behavior:

- A **table** stores a persistent finite relation. At every iteration, the interpreter removes the rows matched by its **remove** clause and inserts those matched by its **add** clause. A **[window a b]** annotation makes the table *metric*: rows are active when they were inserted between *a* and *b* time units in the past ($b = \star$ denotes ∞). A **lagged** table is consumed in the *next* iteration instead of the current one and cannot have a **remove** clause.
- A **let** defines a non-persistent relation within the current time-point. The **filter** keyword signals that the **let** expression may not be finite and can only be used as a boolean condition on top of another relation.
- An **agg let** evaluates its **over** clause, groups the results by **group_by**, and applies the operator *op* (**SUM**, **AVG**, ...) to each group.
- A **rule** matches its **trigger** to current events and, for each match, triggers an action. The action **+** *causes* an event, while **-** *suppresses* it. By default, the action is performed at the current time-point. With **+**, the **[delay n]** annotation triggers the event *proactively* in *n* time-units, while **[next n]** schedules it in *n* time-points.
- A **section** declaration groups all following rules (until the next **section**) into one evaluation block, run either **once** or to a local **fixpoint**, i.e., until the set of events obtained via suppressions and causations is stable.

Each *clause* is an expression consisting of two parts: a disjunction π of *guards* $\kappa_1, \dots, \kappa_n$, with each guard being a conjunction of atoms, and a *filter* **if** φ . Each atom is either

```

event use(int, int, int); // use(c, d, u)
event consent(int, int); // consent(u, c)
event revoke(int, int); // revoke(u, c)
event Sup_Exists0(int, int);
event Sup_Exists2();
// standard library functions elided
table Since1(c:int, u:int) :=
  add {consent(u, c)} remove {revoke(u, c)};
let Exists2() := {use(c, d, u) if !Since1(c, u)};
section fixpoint;
rule +Sup_Exists2() := trigger {if Exists2()};
rule +Sup_Exists0(c, u) :=
  trigger {Sup_Exists2() & use(c, d, u)
    if !Since1(c, u)};
rule -use(c, d, u) :=
  trigger {Sup_Exists0(c, u) & use(c, d, u)};

```

Fig. 2. The EF program enforcing the lawfulness policy φ_{law} of Example 3, generated automatically by Enflash. The S subformula becomes the windowless table `Since1` (consent inserts, revoke removes); the existential block $\exists c, d, u$ arising from $\neg\forall$ is discharged via obligations event `Sup_Exists0` and `Sup_Exists2`. The last rule suppresses `use(c, d, u)` whenever consent for `c` is missing or was revoked.

```

prog ::= evdecl* [pyinit] fun* tfun* item*
evdecl ::= event id(<τ>) ; pyinit ::= pyinit {python}
fun ::= fun id(<c>) : τ {python}
tfun ::= tfun id {python}
item ::= table | let | agg | rule | sec
table ::= [ℓ] [lagged] table id(<c>) [ [window n b] ]
  := add {cl} [remove {cl}];
let ::= [ℓ] [filter] let id(<c>) := {cl};
agg ::= [ℓ] agg let id(<c>) := id(<t>)
  group_by <id> over {cl};
rule ::= [ℓ] rule (+|-) id(<t>) [ [delay n] ] [ [next n] ]
  := trigger {cl};
sec ::= section (fixpoint|once);
τ ::= int | float | str | bool
ℓ ::= @id b ::= n | ★ c ::= id : τ
cl ::= π [if φ] | if φ π ::= κ (or κ)* κ ::= γ (& γ)*
γ ::= id (<id | v>) | id == v t ::= id | const | id (<t>)
φ ::= true | false | id (<t>) | φ & φ | φ | φ | !φ

```

Fig. 3. Grammar of EF programs

a predicate $e(t_1, \dots, t_k)$ or an equality $id == v$. If there is no guard, the clause is reduced to its filter. If there is at least one guard, each free variable of the clause must be the argument of an atom of each guard. This allows guarded clauses to serve as queries over a relation: matching variable assignments are enumerated using the guards, and then filtered.

B. Semantics

An EF program defines the behavior of the μ and ν functions defined in Section II.

The *enforcer state* is a triple (T, T°, Ω) where $T : \mathbb{E} \rightarrow \mathcal{P}(\mathbb{D}^*)$ maps table names to (finite) relations, $T^\circ : \mathbb{E} \rightarrow \mathcal{P}(\mathbb{D}^*)$ buffers the content of lagged tables, and $\Omega \in \mathcal{P}(\mathbb{E} \times \mathbb{D}^* \times (\{\text{ts}, \text{tp}\} \times \mathbb{N}))$ accumulates proactive obligations created via **delay** or **next** together with their deadlines.

After each **fun** declaration associates a function $\hat{f}$ to each function symbol f , where terms are evaluated as in MFOTL. Guards evaluate to sets of valuations under an interpretation $R : \mathbb{E} \rightarrow \mathcal{P}(\mathbb{D}^*)$ of the predicates they contain:

$$\llbracket p(\bar{t}) \rrbracket_R = \{v \mid \llbracket \bar{t} \rrbracket_v \in R(p)\}, \quad \llbracket x == v \rrbracket_R = \{\{x \mapsto v\}\}.$$

Filters are quantifier-free first-order logic expressions, with all variables not occurring in their guard or event head quantified existentially. A clause is evaluated to a set of valuations by joining the guards of each conjunction on their shared variables, taking the union over the disjunction, and filtering:

$$\llbracket \kappa_1 \text{ or } \dots \text{ or } \kappa_m \text{ if } \varphi \rrbracket_R = \left\{ v \in \bigcup_i \bowtie_{\gamma \in \kappa_i} \llbracket \gamma \rrbracket_R \mid v \models_R \varphi \right\}$$

where $\bowtie$ denotes the n -ary join of partial maps.

Programs list **table**, **let**, and **agg** definitions in dependency order. Let P be a program with definitions $d_1, \dots, d_D$ of identifiers $p_1, \dots, p_D$. We evaluate them in turn, extending the interpretation R iteratively and updating the *working set* $S = (T, \tau, D, C, S)$, where S and C denote suppressed and caused events. Formally, for all i , $S(p_i) \triangleq R_i(p_i)$, where

$$R_0 = T \cup (D \setminus S) \cup C \quad R_i = R_{i-1} \cup \{p_i \mapsto \llbracket d_i \rrbracket_{R_{i-1}}\}$$

and the definitions have the following semantics $\llbracket d_i \rrbracket_R$:

$$\begin{aligned}
\llbracket \text{let } p(\bar{x}) := \{c\} \rrbracket_R &= \{v \upharpoonright_{\bar{x}} \mid v \in \llbracket c \rrbracket_R\}, \\
\llbracket \text{table } p(\bar{x}) \text{ [window } n \text{ b]} := add \{c^+\} \text{ remove } \{c^-\} \rrbracket_R \\
&= \left(\mathcal{W}(T(p)) \setminus \{v \upharpoonright_{\bar{x}} \mid v \in \llbracket c^- \rrbracket_R\} \right) \cup \mathcal{W}(\{\tau \cdot v \upharpoonright_{\bar{x}} \mid v \in \llbracket c^+ \rrbracket_R\}) \\
&\text{where } \mathcal{W}(s) = \{v \mid \tau' \cdot v \in s, \tau - \tau' \in [n, b]\}, \\
\llbracket \text{lagged table } p(\bar{x}) \text{ [window } n \text{ b]} := \dots \rrbracket_R &= \mathcal{W}(T(p)), \\
\llbracket \text{agg let } p(\bar{g}, \bar{o}) := g(\bar{s}) \text{ group_by } \bar{g} \text{ over } \{c\} \rrbracket_R \\
&= \{\bar{k} \cdot \bar{w} \mid \bar{k} \in \Pi_{\bar{g}} \llbracket c \rrbracket_R, \bar{w} \in \hat{g} \langle \llbracket \bar{s} \rrbracket_v \mid v \in \llbracket c \rrbracket_R, v(\bar{g}) = \bar{k} \rangle\}.
\end{aligned}$$

Here, $v \upharpoonright_{\bar{x}}$ is the projection $(v(x_1), \dots, v(x_k))$, $\Pi_{\bar{g}} X = \{v(\bar{g}) \mid v \in X\}$ computes group keys, $\langle \dots \rangle$ denotes a multiset, and $\cdot$ concatenates tuples. The $\hat{g}$ and $\hat{w}$ denote functions implementing table (**tfun**) and aggregation operators.

Using the above definitions, we can substitute S for R in the evaluation of clauses and filters, writing $\llbracket c \rrbracket_S$ and $v \models_S \varphi$.

We now define the semantics of rules. A rule r with action $\alpha \in \{+, -\}$, head $p(\bar{t})$, and trigger c collects the matching tuples $\mathcal{A}_S(r) = \{\llbracket \bar{t} \rrbracket_v \mid v \in \llbracket c \rrbracket_S\}$. The obligation set Ω and suppressed/caused sets (C, S) are then updated, appending any newly caused or suppressed event $p(\bar{a})$, $\bar{a} \in \mathcal{A}_S(r)$, to either:

$$\begin{aligned}
&\text{update}(r, \Omega, C, S) \\
&= \begin{cases} (\Omega \cup \{p(\bar{a}), (\text{ts}, \tau + N) \mid \bar{a} \in \mathcal{A}_S(r)\}, C, S) & \text{with } \text{delay } N \\ (\Omega \cup \{p(\bar{a}), (\text{tp}, |\sigma| + N) \mid \bar{a} \in \mathcal{A}_S(r)\}, C, S) & \text{with } \text{next } N \\ (\Omega, C \cup \{p(\bar{a}) \mid \bar{a} \in \mathcal{A}_S(r)\}, S) & \text{if } \alpha = + \\ (\Omega, C, S \cup \{p(\bar{a}) \mid \bar{a} \in \mathcal{A}_S(r)\}) & \text{if } \alpha = -. \end{cases}
\end{aligned}$$

Both μ and ν use the SATURATE loop in Algorithm 2. SATURATE evaluates rules iteratively—once for a **once** section, and until Ω, C, S reaches a fixed point for **fixpoint** sections. The persistent tables are then updated and a new state is returned. The reactive function μ simply calls SATURATE. The ν function first computes the events to be proactively caused at the current timestamp, checking obligations that hit their deadline. If there are any such events, the SATURATE loop is called and the state is updated accordingly.

Algorithm 2 Reactive and proactive functions μ, ν

```

1: function  $\mu((T, T^\circ, \Omega), \sigma, \tau, D)$ 
2:    $(T, S, C, \Omega) \leftarrow \text{SATURATE}((T, \tau, D, \emptyset, \emptyset), T^\circ, \Omega, \sigma)$ 
3:   return  $((T, T^\circ, \Omega), S, C)$ 
4: end function

5: function  $\nu((T, T^\circ, \Omega), \sigma, \tau)$ 
6:    $C \leftarrow \{e \mid (e, (\text{tp}, |\sigma|)) \in \Omega \vee (e, (\text{ts}, t)) \in \Omega\}$ 
7:   if  $C = \emptyset$  else return  $((T, T^\circ, \Omega), \perp)$  end if
8:    $(T, S, C, \Omega) \leftarrow \text{SATURATE}((T, \tau, D, \emptyset, C), T^\circ, \Omega, \sigma)$ 
9:   return  $((T, T^\circ, \Omega), C)$ 
10: end function

11: function  $\text{SATURATE}(S = (T, \tau, D, S, C), T^\circ, \Omega, \sigma)$ 
12:   for all section  $s \in w$  in order do
13:     repeat
14:       for all rule  $r \in s$  do
15:          $(\Omega, S, C) \leftarrow \text{update}(r, \Omega, C, S)$ 
16:       end for
17:     until  $(\Omega, C, S)$  unchanged or  $s$  is a once section
18:   end for
19:   for all table  $p_i(\bar{x}_i) := \text{add } \{c_i^+\} \text{ remove } \{c_i^-\}$  do
20:      $T(p_i) \leftarrow (T(p_i) \setminus \{\tau' \cdot v \upharpoonright_{\bar{x}_i} \mid v \in \llbracket c_i^- \rrbracket_S, \tau' \in \mathbb{N}\}) \cup \{\tau \cdot v \upharpoonright_{\bar{x}_i} \mid v \in \llbracket c_i^+ \rrbracket_S\}$ 
21:   end for
22:   for all lagged table  $p_i(\bar{x}_i) := \text{add } \{c_i\}$  do
23:      $T(p_i) \leftarrow T^\circ(p_i); T^\circ(p_i) \leftarrow \{\tau \cdot v \upharpoonright_{\bar{x}_i} \mid v \in \llbracket c_i \rrbracket_S\}$ 
24:   end for
25:   return  $(T, S, C, \Omega)$ 
26: end function

```

C. Complexity

We bound the runtime of executing a program P in terms of three parameters: the largest number of guards in any single conjunction κ of any clause of P , which we denote w ; the largest number of tuples in any relation $S(p)$ occurring in at least one guard, which we denote N ; and the total number of fixpoint iterations performed, which we denote K . We make the following assumptions that let us isolate relational work:

- (A1) the original trace carries $O(1)$ events per time-point;
- (A2) each scalar function $\hat{f}$, comparison, and aggregation step is $O(1)$, membership tests, insertion, and deletion within a relation is amortized $O(1)$ with hash maps, and any $\hat{\omega}$ on m input rows runs in $O(m)$, returning $O(m)$ rows.

Under (A1) the input-event relations have size $O(1)$, so N is governed by the tables. We prove the following theorems:

Lemma 1. *The boolean $\varphi, v \models_S \varphi$ can be computed in $O(1)$.*

Proof. Since φ has constant size, so it suffices that each atom is $O(1)$. A comparison $t_1 \bowtie t_2$ evaluates its constant-size terms under v in $O(1)$ by (A2). A predicate a let-bound atom $p(\bar{t})$ involves the membership test $\llbracket \bar{t} \rrbracket_v \in S(p)$; using hashsets, this costs $O(1)$. For a table, membership in $\mathcal{W}(T(p))$ can be checked in $O(1)$ by maintaining two separate subtables (timestamps in $\tau - [a, b]$ and in $\tau - [0, a)$) and moving elements between subtables on update. $\square$

Lemma 2. *Given R_{i-1} , computing $\llbracket c \rrbracket_{R_{i-1}}$ for any clause c with all identifiers in the domain of R_{i-1} takes $O(N^w)$ time.*

Proof. Matching a conjunction joins $\leq w$ guards, each ranging over a relation of $\leq N$ tuples. The join enumerates at most N^w valuations, each requiring an $O(1)$ filter by Lemma 1. $\square$

Lemma 3. *Evaluating one rule of P takes $O(N^w)$ time.*

Proof. With memoization, each of the constantly many relations the trigger depends on is computed at most once, in $O(N^w)$ by Lemma 2. Matching the trigger and instantiating its $\leq N^w$ heads are $O(N^w)$. $\square$

Theorem 1. *One call to μ or ν takes $O(K \cdot N^w)$ time.*

Proof. Each of the K iterations is one pass over the $O(1)$ rules of a section, each rule $O(N^w)$ by Lemma 3, hence $O(N^w)$ per iteration. Window updates touch each tuple $O(1)$ times ($O(N)$), committing the layer recomputes constantly many table relations ($O(N^w)$). Each emitted obligation—at most $O(K \cdot N^w)$ per time-point—is discharged once by ν in $O(1)$ by evaluating the validation filter. $\square$

Theorem 2. *A trace of L time-points is processed in $O(L \cdot K_{\max} \cdot N_{\max}^w)$, where $K_{\max}$ and $N_{\max}$ are the largest iteration count and relation size over all time-points.*

Proof. Sum the complexity of Theorem 1 over the L time-points, bounding each factor by its maximum. $\square$

Note that if no tables are used in guards, such as in Example 2, then $N_{\max} = O(1)$ by (A1). If P can be shown to always reach its fixed points in a bounded number of steps, this ensures that the total complexity is $O(L)$, an amortized constant per-step complexity. This is particularly desirable in long-running systems where large enforcer states can accumulate and penalize evaluation latency.

IV. FROM ENFORCEABLE MFOTL TO EF

In this section, we present an algorithm that generates EF programs enforcing formulae in an expressive fragment of MFOTL. Our generation algorithm consists of four components: a normalization procedure that places all temporal subformulae of a formula behind let bindings (IV-A); rewrite rules that extract guards from subformulae (IV-B); rewrite rules that generate enforceable clauses from a formula (IV-C); and a dependency analysis that decides enforceability of a conjunction of clauses and fixes their evaluation order (IV-E).

$$\begin{array}{c}
\text{GRD} \frac{\forall \kappa \in \pi. x \in \text{fv}(\kappa)}{m \vdash (\pi, \varphi) \rightsquigarrow_x^p (\pi, \varphi)} \quad \text{VAC} \frac{}{m \vdash (\pi, \top) \rightsquigarrow_x^+ (\pi, \top)} \\
\\
\text{PRED} \frac{p \in m \quad x \in \bar{t}}{m \vdash (\pi, p(\bar{t})) \rightsquigarrow_x^+ (\{\kappa \wedge p(\bar{t}) \mid \kappa \in \pi\}, \top)} \\
\\
\text{EQ} \frac{}{m \vdash (\pi, x = v) \rightsquigarrow_x^+ (\{\kappa \wedge x = v \mid \kappa \in \pi\}, \top)} \\
\\
\text{NEG} \frac{m \vdash (\pi, \varphi) \rightsquigarrow_x^{-p} (\pi', \varphi')}{m \vdash (\pi, \neg \varphi) \rightsquigarrow_x^p (\pi', \neg \varphi')} \\
\\
\text{AND}^+ \frac{m \vdash (\pi, \varphi_j) \rightsquigarrow_x^+ (\pi', \varphi'_j)}{m \vdash (\pi, \bigwedge_i \varphi_i) \rightsquigarrow_x^+ (\pi', \bigwedge_{i \neq j} \varphi_i \wedge \varphi'_j)} \\
\\
\text{AND}^- \frac{\forall i. m \vdash (\pi, \varphi_i) \rightsquigarrow_x^- (\pi'_i, \varphi'_i)}{m \vdash (\pi, \bigwedge_i \varphi_i) \rightsquigarrow_x^- (\bigcup_i \pi'_i, \bigwedge_i (\pi'_i \rightarrow \varphi'_i))}
\end{array}$$

Fig. 4. Guard extraction rules

A. Let-normal form

In this section, we work with MFOTL formulae rewritten into *let-normal form*. Let-normal form has the shape

let $e_1(\bar{x}_1) = \varphi_1$ in ... let $e_m(\bar{x}_m) = \varphi_m$ in $\square \chi_1 \wedge \dots \wedge \square \chi_n$

where each φ_i is one of ψ , $\bullet \psi$, $\psi_I \psi_r$, or $x \leftarrow \omega(\bar{t}; \bar{g}) \psi$ and ψ is $\exists y_1, \dots, y_k. \chi$, with χ an MFOTL formula without any quantifiers, $\bullet$ and S_I operators, or aggregation/table operators. When a formula is in let-normal form, quantification, temporal operator, and aggregation/table operators are only applied as the last step of evaluating let bindings. One easily shows:

Theorem 3. *For any formula $\square \varphi$ in MFOTL, there exists a semantically equivalent ψ in let-normal form.*

Proof. It suffices to show that any φ in MFOTL, there exists an equivalent let $e_1(\bar{x}_1) = \varphi_1$ in ... let $e_m(\bar{x}_m) = \varphi_m$ in ψ . Rewriting consists of three steps: (i) pull all quantifiers across $\wedge$ and $\neg$ until they are located either at the top of the formula or just below temporal operators, exchange $\exists$ and $\forall$ whenever needed (ii) rewrite $\forall$ as $\neg \exists \neg$ (iii) recursively create one let binding per temporal or aggregation subformula. $\square$

B. Guard extraction

Consider a quantifier-free FOL formula Φ . Next, we show how to derive a disjunction of guards $\pi = \bigvee_i \bigwedge_j \gamma_{ij}$ and a filter φ such that $\pi \wedge \varphi \equiv \Phi$ and $\text{fv}(\Phi) = \text{fv}(\pi)$ for given Φ .

Given a set m of enumerable events that can be used as guards, we define a transformation $m \vdash (\{\kappa_i\}_i, \varphi) \rightsquigarrow_x^+ (\{\kappa'_i\}_i, \varphi')$ that finds a guard for x in $\bigvee_i \kappa_i \wedge \varphi$ and inserts it into the new set of guards $\{\kappa'_i\}_i$, preserving $\bigvee_i \kappa_i \wedge \varphi \equiv \bigvee_i \kappa'_i \wedge \varphi'$. The rules defining $\rightsquigarrow_x$ are shown in Figure 4.

Rule GRD reuses guards already found in π to guard the new variable x . Rules PRED and EQ insert a new guard from an enumerable event and an equation, respectively. VAC provides a trivial base case for $\top$. Rule NEG exchanges $\rightsquigarrow_x^+$ for $\rightsquigarrow_x^-$. Rules AND^+ and AND^- extract guards from conjuncts. The former relies on the equation $\pi \wedge \varphi_j \equiv \pi' \wedge \varphi'_j \implies \pi \wedge \bigwedge_i \varphi_i \equiv \pi' \wedge \bigwedge_{i \neq j} \varphi_i \wedge \varphi'_j$, which enables a guard to be extracted from

any conjunct. The latter uses the dual equation $\forall i. \pi \rightarrow \varphi_i \equiv \pi'_i \rightarrow \varphi'_i \implies \pi \rightarrow \bigwedge_i \varphi_i \equiv \bigvee_i \pi'_i \rightarrow \bigwedge_i (\pi'_i \rightarrow \varphi'_i)$.

Lemma 4. *If $m \vdash (\pi, \varphi) \rightsquigarrow_x^+ (\pi', \varphi')$, then $x \in \bigcap_{\kappa' \in \pi'} \text{fv}(\kappa')$ and $\bigvee_{\kappa \in \pi} \kappa \wedge \pi \equiv \bigvee_{\kappa' \in \pi'} \kappa' \wedge \varphi'$.*

Proof. By induction on the rules in Figure 4. $\square$

Using $\rightsquigarrow_x^+$, we define the transformation Guards that attempts to guard all variables in a set:

$$\begin{aligned}
& \text{Guards}_{\{x_1, \dots, x_k\}}^m(\Phi) \\
&= \begin{cases} (\pi_k, \varphi_k) & \text{iff } m \vdash (\perp, \Phi) \rightsquigarrow_{x_1}^+ \dots \rightsquigarrow_{x_k}^+ (\pi_k, \varphi_k) \\ \perp & \text{otherwise.} \end{cases}
\end{aligned}$$

Lemma 5. *If $\text{Guards}_{\text{fv}(\Phi)}^m(\Phi) = (\pi, \varphi)$, then $\forall \kappa \in \pi. \text{fv}(\Phi) = \text{fv}(\kappa)$ and $\bigvee_{\kappa \in \pi} \kappa \wedge \varphi \equiv \Phi$.*

Proof. By induction on $|\text{fv}(\Phi)|$, observing that $(\pi, \varphi) \rightsquigarrow_x^+ (\pi', \varphi')$ is defined such that $\bigcap_{\kappa \in \pi} \text{fv}(\kappa) \subseteq \bigcap_{\kappa' \in \pi'} \text{fv}(\kappa')$. $\square$

C. Enforcement rewriting

Our main set of rewrite rules turns a formula into a set of *enforcement clauses*. A clause $\theta \Rightarrow \varepsilon$ pairs a trigger $\theta = (\pi, \varphi)$ with an *effect* ε , of the form $e(\bar{t})$ or $\neg e(\bar{t})$, possibly deferred (see the temporal rules below). A clause denotes the EF rule $\square \forall \bar{x}. \theta \rightarrow \varepsilon$ with $\bar{x} = \text{fv}(\theta) \supseteq \text{fv}(\varepsilon)$: for every guarded valuation satisfying the trigger, the effects are applied.

The judgement $\Gamma \vdash \varphi \hookrightarrow^{\mathbb{C}} \mathcal{C}$ produces a set of sets of clauses $\mathcal{C} = \{C_k\}_k$: enforcing any one alternative C_k enforces φ . Dually, the judgement $\Gamma \vdash \varphi \hookrightarrow^{\mathbb{S}} \mathcal{C}$ enforces $\neg \varphi$. The environment Γ maps let-bound event names to a triple of booleans (g, c, s) that indicate whether the corresponding event is guarded, suppressable, and causable, respectively.

Let $\mathcal{C}[f]$ denote $\{\{f(\pi, \psi, \varepsilon) \mid (\pi, \psi) \Rightarrow \varepsilon \in C\} \mid C \in \mathcal{C}\}$ and $\mathcal{C}_1 \otimes \mathcal{C}_2$ denote $\{C_1 \cup C_2 \mid (C_1, C_2) \in \mathcal{C}_1 \times \mathcal{C}_2\}$. The enforcement rewrite rules are shown in Figure 5.

The two EV rules are the only rules generating clauses that modify the trace. If the signature declares e causable ($e \in \mathbb{C}$), then e is enforced by an unconditional clause that causes $e(\bar{t})$; if it declares e suppressable ($e \in \mathbb{S}$), the clause fires exactly when $e(\bar{t})$ is present and suppresses it. The LET rules assert a fresh *obligation event* Cau_e (resp. Sup_e) that will be consumed by clauses that realize e 's body. NEG flips the target formula's polarity: causing $\neg \varphi$ is suppressing φ .

To cause a conjunction ($\text{AND}^{\mathbb{C}}$), every conjunct must be caused, so the clause sets are combined by the product $\otimes$; to suppress it ($\text{AND}^{\mathbb{S}}$) it suffices to falsify one conjunct φ_j , with the other conjuncts added to the filter. To cause $\exists x. \varphi$ ($\text{EX}^{\mathbb{C}}$), it suffices to pick one witness, and the rule simply substitutes a canonical value 0 for x . To suppress it ($\text{EX}^{\mathbb{S}}$), every witness must be falsified, hence x must be guarded.

The temporal rules defer effects. $\text{FUT}_{\Diamond}$ and $\text{FUT}_{\bigcirc}$ apply only when the sub-derivation has reduced each clause trivial triggers; the effect is scheduled at b resp. after n time points.

$$\begin{array}{c}
\text{EV}^C \frac{e \in \mathbb{C}}{\Gamma \vdash e(\bar{t}) \hookrightarrow^C \{ \{ (\top, \top) \Rightarrow e(\bar{t}) \} \}} \quad \text{EV}^S \frac{e \in \mathbb{S}}{\Gamma \vdash \neg e(\bar{t}) \hookrightarrow^S \{ \{ (e(\bar{t}), \top) \Rightarrow \neg e(\bar{t}) \} \}} \quad \text{LET}^C \frac{\Gamma(e)_2 = \top}{\Gamma \vdash e(\bar{t}) \hookrightarrow^C \{ \{ (\top, \top) \Rightarrow \text{Cau}_e(\bar{t}) \} \}} \\
\\
\text{LET}^S \frac{\Gamma(e)_3 = \top}{\Gamma \vdash \neg e(\bar{t}) \hookrightarrow^S \{ \{ (e(\bar{t}), \top) \Rightarrow \text{Sup}_e(\bar{t}) \} \}} \quad \text{NEG} \frac{\Gamma \vdash \varphi \hookrightarrow^{\bar{t}} \mathcal{C}}{\Gamma \vdash \neg \varphi \hookrightarrow^{\bar{t}} \mathcal{C}} \quad \text{AND}^C \frac{\forall i. \Gamma \vdash \varphi_i \hookrightarrow^C \mathcal{C}_i}{\Gamma \vdash \bigwedge_i \varphi_i \hookrightarrow^C \bigotimes_i \mathcal{C}_i} \\
\\
\text{AND}^S \frac{\Gamma \vdash \varphi_j \hookrightarrow^S \mathcal{C}_j}{\Gamma \vdash \bigwedge_i \varphi_i \hookrightarrow^S \mathcal{C}_j[\pi, \psi, \varepsilon \mapsto (\pi, \psi \wedge \bigwedge_{i \neq j} \varphi_i) \Rightarrow \varepsilon]} \quad \text{EX}^C \frac{\Gamma \vdash \varphi \hookrightarrow^C \mathcal{C}}{\Gamma \vdash \exists x. \varphi \hookrightarrow^C \mathcal{C}[\pi, \psi, \varepsilon \mapsto (\pi[0/x], \varphi[0/x]) \Rightarrow \varepsilon[0/x]]} \\
\\
\text{EX}^S \frac{\Gamma \vdash \varphi \hookrightarrow^S \mathcal{C} \quad \mathcal{C}' = \{ C \in \mathcal{C} \mid \forall (\pi, \psi) \Rightarrow \varepsilon \in C. \exists \pi', \psi'. (\pi, \psi) \rightsquigarrow_x^+ (\pi', \psi') \}}{\Gamma \vdash \exists x. \varphi \hookrightarrow^S \{ \{ (\pi', \psi') \Rightarrow \varepsilon \mid (\pi, \psi) \Rightarrow \varepsilon \in C, (\pi, \psi) \rightsquigarrow_x^+ (\pi', \psi') \} \mid C \in \mathcal{C}' \}} \\
\\
\text{FUT}_{\Diamond}^C \frac{\Gamma \vdash \varphi \hookrightarrow^C \mathcal{C} \quad \mathcal{C}' = \{ C \in \mathcal{C} \mid \forall (\pi, \psi) \Rightarrow \varepsilon \in C. \exists p, \bar{t}. \varepsilon = p(\bar{t}) \wedge \pi = \psi = \top \}}{\Gamma \vdash \Diamond_{[a,b]} \varphi \hookrightarrow^C \mathcal{C}'[\pi, \psi, \varepsilon \mapsto (\top, \top) \Rightarrow \Diamond_{[b,b]} \varepsilon]} \quad \text{FUT}_{\bigcirc}^C \frac{\Gamma \vdash \varphi \hookrightarrow^C \mathcal{C} \quad \mathcal{C}' = \{ C \in \mathcal{C} \mid \forall (\pi, \psi) \Rightarrow \varepsilon \in C. \exists p, \bar{t}. \varepsilon = p(\bar{t}) \wedge \pi = \psi = \top \}}{\Gamma \vdash \bigcirc_{[0,b_1]} \dots \bigcirc_{[0,b_n]} \varphi \hookrightarrow^C \mathcal{C}'[\pi, \psi, \varepsilon \mapsto (\top, \top) \Rightarrow \bigcirc \dots \bigcirc \varepsilon]}
\end{array}$$

Fig. 5. Enforcement rewrite rules

D. Generating the candidate clause sets

We write $\text{gate}_p^C(\mathcal{C})$ (resp. gate_p^S) for the set of sets of clauses obtained by conjoining the atom $\text{Cau}_p(\bar{x})$ (resp. Sup_p) to every guard π in $\mathcal{C}$. Rewriting is described in Algorithm 3. The algorithm first processes every let in order, generating the triple $\Gamma(p) = (g, c, s)$ and causation/suppression clauses $\mathcal{C}^C(p), \mathcal{C}^S(p)$, and then rewrites the non-let bound φ^b . The output is a *set of candidate clause sets*; to yield a valid enforcer, a clause set will have to pass the tests in Section IV-E.

TYPELET performs a case split on the let body's outermost operator. Each case decides *guardedness* g and computes the two clause families by rewriting the relevant operand and gating it with p 's obligation event. The boolean c (resp. s) can be computed by checking the emptiness of the set of causation (resp. suppression) clauses. Which clauses are generated depends on the topmost operator of the let. A $\Diamond$ or S is causable only when its interval admits the present ($a = 0$) by causing the (right) operand; a S is suppressable when either $a > 0$ and its left operand is suppressable or both its operands are. A $\bullet$ or aggregation is neither causable or suppressable. Insufficient guarding is fatal for a temporal or aggregation let. For a present let, it only downgrades p to *filter-only*, making it usable as a boolean condition but not caused or suppressed.

Once Γ is complete, GENERATE rewrites φ^b at $\mathbb{C}$, yielding a set of candidate clause sets. REALIZATIONS extends each set of clauses with candidate clause sets from $\mathcal{C}^C(p)$ and $\mathcal{C}^S(p)$: whenever Cau_p (resp. Sup_p) appears in φ^b , it pulls all possible clause sets from $\mathcal{C}^C(p)$ (resp. $\mathcal{C}^S(p)$). This is iterated to a fixpoint since such clauses may themselves reference further obligation events. The resulting candidate clause sets are the input to the dependency analysis in the next section, which selects clause sets that yield a terminating enforcer.

E. Dependency analysis

Not all candidate clause sets generated in the previous step yield a sound enforcer. For example, the set $C_1 = \{(\top, \top) \Rightarrow A(0), (\top, \top) \Rightarrow \neg A(0)\}$ is unsatisfiable, while the set $C_2 =$

Algorithm 3 Generation of candidate clause sets

```

1: function GENERATE( $\square \varphi$ )
2:    $(\mathcal{L}, \varphi^b) \leftarrow \text{LETNORMALFORM}(\square \varphi)$ ;  $\Gamma \leftarrow \emptyset$ 
3:   for all  $(p(\bar{x}) := \psi) \in \mathcal{L}$  do  $\Gamma \leftarrow \text{TYPELET}(\Gamma, p, \bar{x}, \psi)$ 
4:   return  $\bigcup_{C \in \mathcal{C}. \Gamma \vdash \varphi^b \hookrightarrow^C \mathcal{C}} \text{REALIZATIONS}(\Gamma, C)$ 
5: end function
6: function TYPELET( $\Gamma, p, \bar{x}, \psi$ )
7:    $\chi \leftarrow \text{STRIP EXISTS}(\psi)$ ;  $m \leftarrow \{v \mid \exists c, s. \Gamma(v) = (\top, c, s)\}$ 
8:   switch  $\chi$  do
9:     case  $\Diamond_{[a,b]} \varphi$ 
10:       $\theta \leftarrow \text{Guard}_{\text{fv}(\varphi)}^m(\varphi)$ ; if  $\theta = \perp$  then reject
11:       $\mathcal{C}^C(p) \leftarrow a = 0 ? \text{gate}_p^C(\Gamma \vdash \varphi \hookrightarrow^C) : \emptyset$ 
12:     case  $\bullet \varphi \mid \text{Agg}(\dots, \varphi)$ 
13:       $\theta \leftarrow \text{Guard}_{\text{xufv}(\chi)}^m(\varphi)$ ; if  $\theta = \perp$  then reject
14:     case  $\varphi_l S_{[a,b]} \varphi_r$ 
15:       $\theta_l \leftarrow \text{Guard}_{\text{xufv}(\chi)}^m(\varphi_l)$ ;  $\theta_r \leftarrow \text{Guard}_{\text{xufv}(\chi)}^m(\varphi_r)$ 
16:      if  $\theta_l = \perp \vee \theta_r = \perp$  then reject
17:      if  $a = 0$  then
18:         $\mathcal{C}^C(p) \leftarrow \text{gate}_p^C(\Gamma \vdash \varphi_r \hookrightarrow^C)$ 
19:         $\mathcal{C}^S(p) \leftarrow \text{gate}_p^S((\Gamma \vdash \varphi_l \hookrightarrow^S) \otimes (\Gamma \vdash \varphi_r \hookrightarrow^S))$ 
20:      else  $\mathcal{C}^S(p) \leftarrow \text{gate}_p^S(\Gamma \vdash \varphi_r \hookrightarrow^S)$ 
21:      end if
22:     case present  $\chi$ 
23:       $\theta \leftarrow \text{Guard}_{\text{xufv}(\chi)}^m(\chi)$ 
24:      if  $\theta = \perp$  then return  $\Gamma[p \mapsto (\perp, \perp, \perp)]$ 
25:       $\mathcal{C}^C(p) \leftarrow \text{gate}_p^C(\Gamma \vdash \psi \hookrightarrow^C)$ 
26:       $\mathcal{C}^S(p) \leftarrow \text{gate}_p^S(\Gamma \vdash \psi \hookrightarrow^S)$ 
27:   return  $\Gamma[p \mapsto (\top, \mathcal{C}^C(p) \neq \emptyset, \mathcal{C}^S(p) \neq \emptyset)]$ 
28: end function

```

$\{(A(x), \top) \Rightarrow A(x+1)\}$ generates an infinite number of events and never terminates. We present a combination of two checks that guarantees that a set of clauses can be finitely enforced. The first rules out events that are simultaneously caused and suppressed, as in our first example; the second rules out non-terminating fixpoints, as in the second example.

a) *Cause/suppress conflicts*: Consider the *Event Dependency Graph* (EDG) whose vertices are base events (let atoms are decomposed the events defining them), and there is an edge

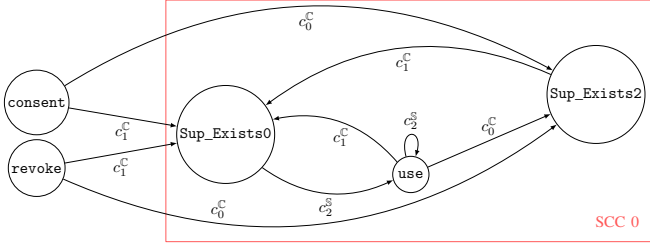


Fig. 6. EDG for the lawfulness policy of Figure 2 (our running example). Existentials are discharged via two obligation events `Sup_Exists0/2`, which, together with `use`, form a three-node SCC. As `use` is only *suppressed* (c_2^S) and `Sup_Exists0/2` is only *caused* ($c_{0,1}^C$), no event is both caused and suppressed, so the conflict check passes.



Fig. 7. Data-flow graph over argument positions. The self-loop on `use.1` ($\text{use}(c, d, u) \Rightarrow \neg \text{use}(c, d, u)$, our running example) is stable and accepted; the self-loop on `A.0` ($A(x) \Rightarrow A(x+1)$) passes through a non-stable term and is rejected.

$e \xrightarrow{c_k^a} e'$ when e occurs in the trigger of some clause k and e' occurs in its effect with polarity $a \in \{\mathbb{C}, \mathbb{S}\}$ (Figure 6). Its strongly connected components (SCCs) are mutually-influencing event sets. For each e that one clause with trigger θ_C causes and another clause with trigger θ_S suppresses, the pair (θ_C, θ_S) is sent to an SMT solver, which tests the unsatisfiability of $\theta_C \wedge \theta_S$ by only taking into account events from SCCs (strictly) before e in the topological order of SSCs. Any SAT result rejects the corresponding clause. This rules out C_1 above but allows, e.g., $\{(\top, B(0)) \Rightarrow A(0), (\top, \neg B(0)) \Rightarrow \neg A(0)\}$, which is equivalent to $\Box A(0) \leftrightarrow B(0)$.

b) Termination: Termination is decided on a distinct *Data-Flow Graph* (DFG) whose nodes are event arguments $e.i$ and that contains an edge $e.i \rightarrow e'.j$ for any clause whose trigger mentions $e(\dots, t_i = x, \dots)$ and whose effect is $e'(\dots, t'_j, \dots)$ with x occurring in t'_j . An edge is *stable* iff all function symbols in t'_j are stable. Enforcement terminates iff no non-stable edge lies on a cycle. As a result, causing a fresh value on every iteration (e.g. $(A(x), \top) \Rightarrow A(x+1)$) is rejected, while, e.g., $(\text{use}(d), \top) \Rightarrow \neg \text{use}(d)$ is accepted because no new domain values are generated (Figure 7).

F. Compilation

Consider a closed formula $\Box \varphi$, an environment Γ as computed in Section IV-D, and a candidate clause set R that fulfills both checks in Section IV-E. Fix a reverse-topological order $\prec$ on the condensed SCCs of its EDG, which also induces an order on clauses. The compilation algorithm in Algorithm 4 turns (Γ, R) into an EF program that enforces $\Box \varphi$.

Each clause becomes an EF rule; a caused effect yields a $+$ rule, a suppressed effect a $-$ rule, and a deferred effect

Algorithm 4 Compilation algorithm

```

1: function COMPILE( $\Gamma, R, \prec$ )
2:    $P \leftarrow$  declaration of all base/obligation events in  $R$ 
3:   for all  $p \in \Gamma$  do
4:     append to  $P$  the item realizing  $p$ 's body: table (past
       operator), agg, or filter let (non-guarded present) or let
5:   end for
6:   for all  $(\pi, \psi) \Rightarrow \varepsilon \in R$ , by SCC in order  $\prec$  do
7:     open a section for the SCC if not already open
8:     append the rule  $[\pm] \hat{\varepsilon} := \text{trigger} \{ \pi \text{ if } \psi \}$ ,
       where  $\hat{\varepsilon}$  carries a  $[\text{delay } n]$  (resp.  $[\text{next } n]$ ) annotation if  $\varepsilon$  is
        $\Diamond_{[n,n]} p(\dots)$  (resp.  $\bigcirc \cdot \bigcirc p(\dots)$  with  $n \bigcirc$ )
9:   end for
10:  return  $P$ 
11: end function

```

a proactive **delay/next** causation. Each let recorded in Γ becomes a **table**, **let**, **agg** or **filter** item. The SCC order $\prec$ provides the EF sections.

Example 4. Running *EnfGuard* on φ_{law} (Example 3) yields the program of Figure 2. Let-normalization binds the since subformula $\neg \text{revoke}(u, c) \text{ S } \text{consent}(u, c)$ to a let, which TYPELET compiles to the windowless table `Since1`: its right operand `consent` populates the **add** clause and its left operand `revoke` the **remove** clause. The whole quantifier block is kept together, so the negated body $\exists c, d, u. \text{use}(c, d, u) \wedge \neg \text{Since1}(c, u)$ is bound to the Boolean let `Exists0`. Suppressing it yields three clauses in one **fixpoint** section: $+\text{Sup_Exists0/2}$ raises the obligation event whenever a violation is present, and $-\text{use}$ then suppresses each offending $\text{use}(c, d, u)$ for which `consent` is missing or has been revoked. The resulting EDG (Figure 6) has a two-node SCC whose only suppressed event is `use`, so the conflict check passes; the only data-flow cycle is the stable self-loop $\text{use}(c, d, u) \Rightarrow \neg \text{use}(c, d, u)$ (Figure 7, left), so termination holds.

Theorem 4 (Compilation correctness). *Let R be a candidate clause set accepted by the two checks of Section IV-E and $P = \text{COMPILE}(\Gamma, R, \prec)$. Then P is a sound enforcer for $\Box \varphi$.*

Proof sketch. The argument has two parts: the rewrite rules are sound, and the two checks ensure fixpoint termination.

Local soundness. A straightforward induction on Figure 5 shows that, whenever Γ faithfully records each let's capabilities, every $C \in \mathcal{C}$ with $\Gamma \vdash \varphi \hookrightarrow^C \mathcal{C}$ enforces φ once its clauses have been applied, and dually $\hookrightarrow^S$ falsifies φ . Hence, if the EF program terminates, it soundly enforces $\Box \varphi$.

Well-definedness and termination. The cause/suppress check rejects any R in which an event could be caused and suppressed in the same iteration. The data-flow check ensures that only finitely many new events can be caused or suppressed within each iteration. Hence, every fixed point is reached in finitely many iterations. $\square$

V. EVALUATION

We implement the compilation pipeline of Section IV and the EF interpreter of Section III in a tool called *Enfflash*.

The compiler (7k LOC of OCaml) parses an MFOTL specification, runs let-normalization, guard extraction, enforcement rewriting, and the dependency analysis, discharges the cause/suppress checks with Z3, and emits an EF program. The interpreter (4k LOC of Rust) loads the EF program, computes a bound on its complexity as in Section III, and exposes the reactive and proactive entry points μ, ν . Rust’s `pyo3` Python bindings are used to evaluate user-defined functions. Enfflash is free software and available online.¹

Our evaluation answers four research questions:

- RQ1 (Expressiveness) Does Enfflash support the specifications of existing benchmarks and real applications?
- RQ2 (Latency) How does Enfflash’s per-event latency compare to state-of-the-art first-order enforcers?
- RQ3 (Overhead in web applications) Is the end-to-end enforcement overhead small enough for user-facing operations in real web applications?
- RQ4 (Comparison with permission engines) How does Enfflash compare to a state-of-the-art non-temporal permission engine (Cedar) on access control, accounting for the cost of maintaining temporal state?

A. Experimental setup

All experiments run under Ubuntu 24.04.3 on an Intel Core Ultra 7 165U with 12 cores and 32GB of RAM. We run two types of experiments: tool-only benchmarks, where a trace from an existing benchmark is replayed against enforcement tools; and application benchmarks, where those tools serve as Policy Decision Points (PDP) for web applications [13].

In tool-only benchmarks, latency is measured as wall-clock time around the processing of single time points. We report per-time-point average and maximum latency for each run.

In application benchmarks, we measure full round-trip page latency using Python’s `requests` library. against a local server with the ‘performance’ CPU governor activated.

B. Tool-only: the EnfGuard suite (RQ1,2)

We first evaluate ENFFLASH on EnfGuard’s 6 enforcement benchmarks, which span data protection (GDPR, FUN), aggregation-heavy policies (AGG), fault clustering (CLUSTER), infrastructure monitoring (IC) and database updates in a large system (NOKIA). We use all formulae in the benchmark with the largest available log. For each, we compile the specification and replay the provided traces, measuring latency against Enfflash, EnfGuard [4], Enfpoly [8], and Monpoly [14] on identical inputs, with 3 repetitions. We selected EnfGuard and Enfpoly as the two fastest available MFOTL enforcers. Monpoly is an optimized MFOTL *monitor*, i.e., a tool that identifies, but does not correct, policy violations; while Monpoly is therefore not a direct competitor to enforcement tools, it provides a challenging state-of-the-art latency baseline for a slightly simpler problem.

TABLE I
LATENCY IN MS—*mean (max)*—ON THE LARGEST LOGS ON THE ENFGUARD BENCHMARK SUITE AT MAXIMAL REAL-TIME ACCELERATION

Policy	Enfflash	Monpoly	Enfpoly	EnfGuard
GDPR (<i>timeout = 60 s</i>)				
consent	0.10 (2)	0.14 (2)	0.11 (1)	1.32 (6)
deletion	0.09 (1)	0.09 (1)	—	0.32 (3)
gdpr	0.09 (1)	0.11 (3)	—	3.97 (13)
information	0.09 (2)	0.09 (2)	—	1.20 (7)
lawfulness	0.09 (1)	0.10 (1)	0.09 (2)	1.46 (7)
limitation	0.09 (2)	—	—	0.35 (59)
sharing	0.09 (1)	0.10 (1)	—	1.14 (5)
FUN (<i>timeout = 60 s</i>)				
consent	0.10 (2)	—	—	0.83 (5)
consent_management	0.09 (1)	—	—	0.29 (2)
deletion	0.09 (1)	—	—	0.32 (3)
gdpr	0.11 (1)	—	—	3.36 (12)
information	0.09 (1)	—	—	1.55 (9)
sharing	0.09 (1)	—	—	1.23 (5)
CLUSTER (<i>timeout = 60 s</i>)				
dbscan	3.82 (10)	—	—	t.o.
grubbs	2.98 (9)	—	—	t.o.
AGG (<i>timeout = 60 s</i>)				
p1	0.98 (3)	0.19 (3)	—	t.o.
p2	0.84 (3)	0.32 (3)	—	t.o.
p3	0.94 (2)	0.39 (3)	—	t.o.
p4	1.96 (4)	0.52 (3)	—	t.o.
p5	1.23 (3)	0.21 (2)	—	t.o.
p6	0.74 (2)	0.44 (3)	—	t.o.
NOKIA (<i>timeout = 120 s</i>)				
del-1-2	0.22 (3)	0.20 (19)	—	t.o.
del-2-3	0.23 (5)	0.12 (3)	—	t.o.
del-3-2	0.22 (3)	0.13 (3)	—	t.o.
delete	0.21 (3)	0.13 (4)	0.13 (3)	0.79 (11)
ins-1-2	t.o.	1.06 (112)	—	t.o.
ins-2-3	0.32 (4)	0.41 (9)	—	t.o.
ins-3-2	0.35 (5)	0.40 (7)	—	t.o.
insert	0.25 (5)	0.20 (5)	0.20 (3)	1.30 (25)
script1	1.07 (15)	0.28 (3)	0.15 (2)	t.o.
select	0.22 (2)	0.13 (3)	0.12 (3)	0.82 (12)
update	0.23 (3)	0.13 (4)	0.13 (4)	0.85 (15)
IC (<i>timeout = 600 s</i>)				
block_validation_latency	0.23 (4)	t.o.	—	0.15 (7)
clean_logs	0.14 (6)	0.12 (5)	—	t.o.
finalization_consistency	0.09 (4)	0.08 (4)	—	t.o.
finalized_height	t.o.	t.o.	—	0.17 (5)
logging_behavior	0.70 (94)	t.o.	—	0.11 (25)
reboot_count	0.10 (4)	0.11 (4)	—	t.o.
replica_divergence	0.09 (4)	0.08 (4)	—	0.23 (5)
unauthorized_connections	0.42 (71)	t.o.	—	t.o.

Results (Table I): Every dash denotes that the formula is not supported by the corresponding tool. Enfflash supports all policies in the benchmarks, with only the NOKIA formulae requiring rewriting to avoid the nesting of past and future operators in guards. Enfflash’s average latency is 3–30× lower than EnfGuard’s and broadly at par with Monpoly’s (except for 2 formulae in IC), with Enfflash 1–3× faster than the monitor on GDPR and FUN, Monpoly 1–3× faster on AGG

¹ • URL

TABLE II
GDPRSOCIAL PAGE LATENCY IN MS (+OVERHEAD TO NON-ENFORCED
BASELINE) FOR REPRESENTATIVE APPLICATION VIEWS

View	$n = 10^2$		$n = 10^3$		$n = 10^4$		$n = 10^5$	
View timeline	29.15	+18	21.69	+7	26.86	+12	26.33	+15
Post message	20.23	+18	12.23	+9	18.25	+14	19.77	+15
Grant consent	6.43	+3	9.23	+6	10.48	+6	10.78	+7
Revoke consent	6.01	+3	6.54	+3	6.92	+3	6.65	+4
View privacy notices	8.57	+3	7.95	+2	8.23	+4	9.06	+3
Request personal data	8.78	+4	9.21	+6	7.04	+2	11.70	+7

and NOKIA, and Enfflash generally faster on IC. Average (resp. max) latency varies between 0.1–4 ms (resp. 1–94 ms).

C. Application: GDPR-compliant social network (RQ1,3)

Our first deployment instruments GDPRSOCIAL, a microblogging app implemented in ca. 2.5k LOC of Python/Django that runs against an extensive GDPR specification encoded as a 400-line MFOTL formula. Our GDPR specification is intrinsically *temporal*, with both statefulness (consent and revocation thereof must be persisted) and proactivity (personal data must be accessed, rectified, or deleted in a timely manner when users require it). Enfflash acts as an enforcement backend invoked through the Instrlib [13] library. We measure the latency of a set of representative application views: accessing a user’s timeline, posting a message, giving or revoking consent for personalized advertisement, viewing privacy notices, and requesting access to one’s data. Measurements are performed for $n \in \{10^2, 10^3, 10^4, 10^5\}$ users and $100n$ messages over 10 repetitions after 5 warm-up calls. As a baseline, we perform measurements on a copy of the app stripped of all enforcement. Note that due to the temporal nature of the policy and the much higher latency of EnfGuard, no systematic experiments could be performed with competing tools.

Results (Table II): Enfflash supports the large input formula. All views load in 6–30 ms, with enforcement inducing an overhead of 3–18 ms with respect to the non-enforced baseline. This loading time would typically be perceived as instantaneous by users [15, p. 135]. In particular, runtime latency does not grow with n , and hence enforcement scales well with database size, a key requirement for a social network.

D. Application: event-management platform (RQ1,3,4)

Our second deployment is EVENTMANAGER, an event-management platform whose access-control policy mixes role-based rules with stateful constraints (e.g., only admins may add moderators, only owners may promote an attendant of an event to manager of an event). This application was proposed by Basin et al. [16] and implemented in both Cedar [7] and ν ActionGUI [17] authorization engines. Because most of this policy is expressible without temporal operators when the roles are fixed, it admits a direct encoding in these two platforms, giving us two non-temporal baselines. However, the fact that these roles can change over time means that an extensive dynamic context must be passed to the non-temporal

TABLE III
EVENTMANAGER PAGE LATENCY IN MS: ENFFLASH (E) VS. CEDAR (C)

View	$n = 1$		$n = 10$		$n = 100$	
	E	C	E	C	E	C
View events	33.08	1 s	1 s	11 s	14 s	112 s
View profile	36.85	4 s	37.26	3 s	37.69	3 s
Create event	22.41	32.48	21.62	24.73	16.22	97.20
Grant consent	19.01	14.72	25.64	13.04	25.58	41.53
Revoke consent	6.12	5.35	6.44	4.96	7.31	4.79
Analyze event	10.27	119.38	27.19	112.52	19.13	113.15

TABLE IV
LATENCY IN μ S ON CEDAR BENCHMARKS: ENFFLASH (E) VS. CEDAR
EVALUATION (C^e) VS. CEDAR INITIALIZATION+EVALUATION (C^f)

Benchmark	$n = 25$			$n = 50$			$n = 75$			$n = 100$		
	E	C^e	C^f	E	C^e	C^f	E	C^e	C^f	E	C^e	C^f
GDrive	6	5	451	12	11	1 ms	13	18	5 ms	25	28	12 ms
GitHub	54	12	702	158	18	2 ms	204	28	6 ms	267	45	17 ms
TinyTodo	12	7	776	13	9	1 ms	15	14	3 ms	13	19	4 ms

evaluator at every request, potentially increasing the latency. We straightforwardly translated Nguyễn et al.’s policy into ca. 150 lines of MFOTL by replacing all the predicates of the form $\text{hasRole}(\bar{x})$ by $\neg \text{leftRole}(\bar{x}) \vee \text{becomeRole}(\bar{x})$. Measurements are performed for $n \in \{1, 10, 100\}$ users and $10n$ events.

a) Results (Table III): Our Enfflash implementation is faster than the Cedar baseline on 4 of 6 tested views and achieves similar performance on the other 2. In particular, Enfflash provides a 8–30 $\times$ speedup on the ‘view events’ page and a 80–110 $\times$ speedup on ‘view profile’ page, both of which query permissions about a large number of entities.

E. Tool-only: Cedar benchmarks with dynamic contexts (RQ4)

A non-temporal engine such as Cedar evaluates a request against a policy and a fixed set of entities. To enforce a policy that depends on past actions, the surrounding application must reconstruct and reload these entities whenever a relevant action is performed. This scenario is *not* the focus of Cedar’s developers, but can occur in many practically relevant applications. We show how Enfflash compares to Cedar in this scenario, we compare both tools on Cedar’s three benchmarks [7], measuring both (i) Cedar’s evaluation time (denoted C^e) and (ii) the time to rebuild and reload the changed state (denotes C^f), against Enfflash latency (denoted E) when it updates its state incrementally. For each benchmark, we measure per-event latency for a memory with $n \in \{25, 50, 75, 100\}$ entities.

a) Result (Table IV): Per-event, Enfflash is generally 1–14 $\times$ slower than Cedar as an evaluation engine. Nevertheless, Enfflash can still deliver similar performance as Cedar on certain benchmarks as the number of entities grow. Moreover, Enfflash is consistently two orders of magnitude faster than Cedar’s full reload and evaluation pipeline. The more state changes occur, the more the advantage of Enfflash’s stateful approach over Cedar’s static evaluator becomes evident.

F. Summary

Enfflash supports all policies in the EnfGuard suite and both application benchmarks, suggesting that the supported fragment of MFOTL is expressive enough to support a broad range of use cases, including with very large policies (RQ1). Our tool achieves average per-event latency of 0.1–4 ms, consistently 3–30× lower than the fastest MFOTL enforcer to-date (EnfGuard) and broadly comparable to Monpoly, a monitor solving a strictly simpler problem (RQ2). In the GDPRSOCIAL application, enforcement adds 3–18 ms of overhead despite the policy’s complexity, and this overhead does not significantly grow with database size, keeping all views near-instantaneous (RQ3). Enfflash matches or significantly improves on Cedar’s performance in our EVENTMANAGER application and is two orders of magnitude faster than Cedar’s full reload-and-evaluate pipeline in dynamic contexts. While still providing sub-ms latency, Enfflash remains 1–14× slower than Cedar’s evaluator on some static policies (RQ4).

VI. RELATED WORK

Runtime enforcement was introduced by Schneider in his seminal work on security automata [1]. Enforcement initially focused on execution cutting, i.e., terminating a system to prevent violations. Suppression [2], [18] and edit automata [19] were later introduced to allow for suppression and causation of events, respectively. However, these enforcers were still allowed to (unrealistically) buffer events before making decisions. This capability was later dropped [20], and other capabilities, such as delaying events [21], [22] and code inspection [23], were considered. In parallel, fine-grained enforcer capabilities were introduced, distinguishing between causable or suppressable and only-observable events [24]. Time-aware enforcers rely on timed automata either as both a policy language and mechanism [25]–[28] or as the translation target for higher-level logics such as MITL [29]. Basin et al. [11] describe a proactive enforcer for dynamic condition response graphs [30], a propositional formalism.

The enforcement of *first-order temporal* policies dates back to Hallé and Villemare [31] for LTL-FO⁺, a first-order variant of future-only LTL. Hublet et al. [8] enforce a restricted set of MFOTL policies that cannot refer to the future. They later extend their work to proactive enforcement [5] and optimize and extend their enforcer [4]. Aceto et al. [32], [33] consider policies in first-order Hennessy-Milner Logic with recursion, but their approach is non-metric and without causation.

Runtime enforcement as presented here is generic and thus provides theoretical underpinning for tools in many specialized domains, like access and usage control [7], [17], Android permission management [34], IoT [35], and LLM agent security [36], modulo appropriate instrumentation [13].

VII. CONCLUSION

We presented Enfflash, a first-order enforcer that sets a new state of the art in terms of runtime performance, reducing overhead 3–20× and supporting integration in user-facing apps. A key question to be addressed in future work concerns

the parallelization of enforcement in large applications, for which a single enforcer yields an impractical bottleneck. Fortunately, mature approaches to parallelized monitoring already exist [37]. Preliminary experiments inspired by such approaches—e.g., splitting the enforcement of the GDPR policy in Section V-C by user—showed promising results, which we could not report in this paper for space reasons.

REFERENCES

- [1] F. B. Schneider, “Enforceable security policies,” *ACM Trans. Inf. Syst. Secur. (TISSEC)*, vol. 3, no. 1, pp. 30–50, 2000.
- [2] L. Bauer, J. Ligatti, and D. Walker, “More enforceable security policies,” in *Workshop on Foundations of Computer Security (FCS)*, 2002.
- [3] J. Chomicki, “Efficient checking of temporal integrity constraints using bounded history encoding,” *ACM Transactions on Database Systems (TODS)*, vol. 20, no. 2, pp. 149–186, 1995.
- [4] F. Hublet, L. Lima, D. Basin, S. Krstić, and D. Traytel, “Scaling up proactive enforcement,” in *37th International Conference on Computer Aided Verification (CAV)*, ser. LNCS, R. Piskac and Z. Rakamaric, Eds., vol. 15933. Springer, 2025, pp. 370–392.
- [5] —, “Proactive real-time first-order enforcement,” in *36th International Conference on Computer Aided Verification (CAV)*, ser. LNCS, A. Gurfinkel and V. Ganesh, Eds., vol. 14682. Springer, 2024, pp. 156–181.
- [6] F. Hublet, D. Basin, and S. Krstić, “Enforcing the GDPR,” in *28th European Symposium on Research in Computer Security (ESORICS)*, ser. LNCS, G. Tsodik, M. Conti, K. Liang, and G. Smaragdakis, Eds., vol. 14345. Springer, 2023, pp. 400–422.
- [7] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. H. Kastner, A. Mamat, D. McAdams, M. McCutchen, N. Rungta, E. Torlak, and A. Wells, “Cedar: A new language for expressive, fast, safe, and analyzable authorization,” *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA1, pp. 670–697, 2024.
- [8] F. Hublet, D. Basin, and S. Krstić, “Real-time policy enforcement with metric first-order temporal logic,” in *27th European Symposium on Research in Computer Security (ESORICS)*, ser. LNCS, V. Atluri, R. D. Pietro, C. D. Jensen, and W. Meng, Eds. Springer, 2022, pp. 211–232.
- [9] A. Pnueli, “The temporal logic of programs,” in *18th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1977, pp. 46–57.
- [10] R. Koymans, “Specifying real-time properties with metric temporal logic,” *Real-time systems*, vol. 2, no. 4, pp. 255–299, 1990.
- [11] D. Basin, S. Debois, and T. Hildebrandt, “Proactive enforcement of provisions and obligations,” *Journal of Computer Security*, vol. 32, no. 3, pp. 247–289, 2024.
- [12] K. Havelund, M. Omer, and D. Peled, “Operational and declarative runtime verification (keynote),” in *7th ACM International Workshop on Verification and Monitoring at Runtime Execution (VORTEX)*, D. Ancona and G. Audrito, Eds. ACM, 2024, pp. 3–12.
- [13] F. Hublet, D. Basin, L. Hu, S. Krstić, and L. Reese, “Instrumenting runtime enforcement,” in *25th International Conference on Runtime Verification (RV)*, ser. LNCS, vol. 16087. Springer, 2025, pp. 160–180.
- [14] D. Basin, M. Harvan, F. Klaedtke, and E. Zălinescu, “MONPOLY: Monitoring usage-control policies,” in *2nd International Conference on Runtime Verification (RV)*, ser. LNCS, vol. 7186. Springer, 2011, pp. 360–364.
- [15] J. Nielsen, *Usability engineering*. AP Professional, 1993.
- [16] D. Basin, J. Guarnizo, S. Krstić, H. Nguyen, and M. Ochoa, “Is modeling access control worth it?” in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, W. Meng, C. D. Jensen, C. Cremers, and E. Kirda, Eds. ACM, 2023, pp. 2830–2844.
- [17] S. Krstić, H. Nguyen, and D. Basin, “Model-driven privacy,” *Proc. Priv. Enhancing Technol.*, vol. 2024, no. 1, pp. 314–329, 2024.
- [18] M. Fredrikson, R. Joiner, S. Jha, T. W. Reps, P. A. Porras, H. Saïdi, and V. Yegneswaran, “Efficient runtime policy enforcement using counterexample-guided abstraction refinement,” in *24th International Conference on Computer Aided Verification (CAV)*, ser. LNCS, P. Madhusudan and S. A. Seshia, Eds., vol. 7358. Springer, 2012, pp. 548–563.
- [19] J. Ligatti, L. Bauer, and D. Walker, “Edit automata: Enforcement mechanisms for run-time security policies,” *International Journal of Information Security*, vol. 4, pp. 2–16, 2005.

- [20] M. Ngo, F. Massacci, D. Milushev, and F. Piessens, “Runtime enforcement of security policies on black box reactive programs,” in *42nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, S. K. Rajamani and D. Walker, Eds. ACM, 2015, pp. 43–54.
- [21] S. Pinisetty, Y. Falcone, T. Jérón, H. Marchand, A. Rollet, and O. Nguena Timo, “Runtime enforcement of timed properties revisited,” *Formal Methods Syst. Des.*, vol. 45, pp. 381–422, 2014.
- [22] Y. Falcone, T. Jérón, H. Marchand, and S. Pinisetty, “Runtime enforcement of regular timed properties by suppressing and delaying events,” *Science of Computer Programming*, vol. 123, pp. 2–41, 2016.
- [23] S. Pinisetty, V. Preoteasa, S. Tripakis, T. Jérón, Y. Falcone, and H. Marchand, “Predictive runtime enforcement,” *Formal Methods Syst. Des.*, vol. 51, no. 1, pp. 154–199, 2017.
- [24] D. Basin, V. Jugé, F. Klaedtke, and E. Zălinescu, “Enforceable security policies revisited,” *ACM Trans. Inf. Syst. Secur. (TISSEC)*, vol. 16, no. 1, pp. 1–26, 2013.
- [25] Y. Falcone, S. Krstić, G. Reger, and D. Traytel, “A taxonomy for classifying runtime verification tools,” *Int. J. Softw. Tools Technol. Transf.*, vol. 23, no. 2, pp. 255–284, 2021.
- [26] Y. Falcone and S. Pinisetty, “On the runtime enforcement of timed properties,” in *19th International Conference on Runtime Verification (RV)*, ser. LNCS, B. Finkbeiner and L. Mariani, Eds., vol. 11757. Springer, 2019, pp. 48–69.
- [27] S. Pinisetty, Y. Falcone, T. Jérón, and H. Marchand, “TiPEX: A tool chain for timed property enforcement during execution,” in *International Conference on Runtime Verification (RV)*. Springer, 2015, pp. 306–320.
- [28] M. Renard, A. Rollet, and Y. Falcone, “GREP: games for the runtime enforcement of properties,” in *International Conference on Testing Software and Systems (ICTSS)*, ser. LNCS, N. Yevtushenko, A. Cavalli, and H. Yenigün, Eds., vol. 10533. Springer, 2017, pp. 259–275.
- [29] O. Maler, D. Ničković, and A. Pnueli, “From MITL to timed automata,” in *International conference on formal modeling and analysis of timed systems (FORMATS)*, ser. LNCS, E. Asarin and P. Bouyer, Eds., vol. 4202. Springer, 2006, pp. 274–289.
- [30] T. Hildebrandt, R. R. Mukkamala, T. Slaats, and F. Zanitti, “Contracts for cross-organizational workflows as timed dynamic condition response graphs,” *The Journal of Logic and Algebraic Programming*, vol. 82, no. 5-7, pp. 164–185, 2013.
- [31] S. Hallé and R. Villemaire, “Runtime enforcement of web service message contracts with data,” *IEEE Trans. Serv. Comput.*, vol. 5, no. 2, pp. 192–206, 2012.
- [32] L. Aceto, I. Cassar, A. Francalanza, and A. Ingólfssdóttir, “Bidirectional runtime enforcement of first-order branching-time properties,” *Logical Methods in Computer Science*, vol. 19, 2023.
- [33] —, “On first-order runtime enforcement of branching-time properties,” *Acta Informatica*, pp. 1–67, 2023.
- [34] A. Sadeghi, R. Jabbarvand, N. Ghorbani, H. Bagheri, and S. Malek, “A temporal permission analysis and enforcement framework for Android,” in *40th International Conference on Software Engineering (ICSE)*, M. Chaudron, I. Crnkovic, M. Chechik, and M. Harman, Eds. ACM, 2018, pp. 846–857.
- [35] Z. B. Celik, G. Tan, and P. D. McDaniel, “IoTGuard: Dynamic enforcement of security and safety policy in commodity iot,” in *26th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2019.
- [36] H. Wang, C. M. Poskitt, and J. Sun, “AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents,” in *48th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 2026.
- [37] J. Schneider, D. Basin, F. Brix, S. Krstić, and D. Traytel, “Scalable online first-order monitoring,” *Int. J. Softw. Tools Technol. Transf.*, vol. 23, no. 2, pp. 185–208, 2021.